



نظري

6

19

2280 SP

كلية الهندسة المعلوماتية

السنة الثالثة

Scan
Conversion



د. عمار النحاس

محتوى مجاني غير مخصص للبيع التجاري

RB Informatics; 09/09/2020

بيانات حاسوبية

تحدثنا في المحاضرة السابقة عن كيفية التحويل للشكل النظامي لمنطقة العرض (Canonical Viewing Volume) وستابع عن كيفية القيام بعمليات ال Clipping.

إن عملية ال Clipping تتم على primitives أي على الخطوط والمضلعات.

إذا ماهي الخوارزميات التي تتم من خلالها عمليات ال clipping؟

سندرس خوارزمية ال clipping لقطعة مستقيمة لأن خوارزميات clipping لل Polygons مشتقة منها .

نستطيع عن طريقه هذه الخوارزميات معرفة إذا كانت داخل منطقة العرض أو خارجها أو جزء منها داخل منطقة العرض نتعامل مع قطعة مستقيمة وليس مستقيم لأن القطعة المستقيمة لها نقطة بداية ونقطة نهاية، لكن حساب تقاطع قطعتين مستقيمتين ليس سهلاً فقد يكون التقاطع بين حامل كل من هذه القطع لكن القطعتان غير متقاطعتين ، إذا نحتاج لطريقة تسهل هذه العملية.

خوارزمية Cohen-Sutherland

1001	1000	1010
0001	0000	0010
0101	0100	0110

نقوم بتحديد حواف نافذة العرض لينتج أربعة خطوط

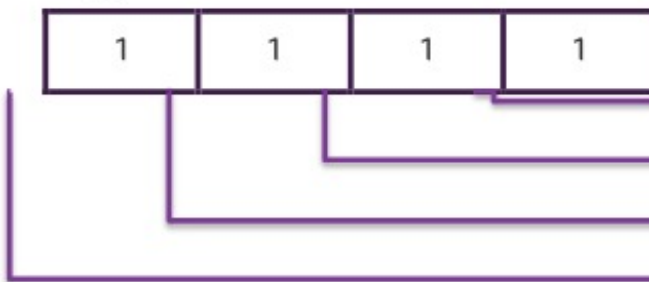
نسمي كل منها (up-down-left-right) حسب مكان الخط.

ينتج تسعة أقسام نضع لكل قسم bit code-4

ولكل بت من البتات الأربعة دلالة على جهة من جهات نافذة

الرؤيا وبذلك فإننا، بقراءة البت نستطيع معرفة مكان

القسم الموافق من النافذة.



وتكون دلالات البتات الأربعة كالآتي:

- إذا كانت النقطة يسار منطقة العرض
- إذا كانت النقطة يمين منطقة العرض
- إذا كانت النقطة أسفل منطقة العرض
- إذا كانت النقطة أعلى منطقة العرض

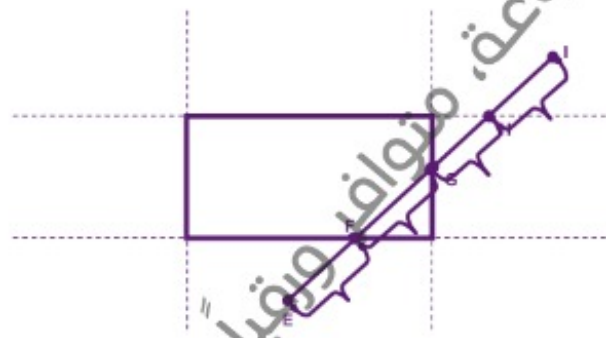
مثلاً المنطقة 1001 تقع أعلى يسار الشاشة لأن قيمة كل من البت الأول والرابع هو 1

إذاً بداية ونهاية القطعة المستقيمة ستكون في إحدى هذه المناطق، لذا نحسب الرمز بكل من بداية ونهاية القطعة المستقيمة لنعرف إذا كانت تمر من نافذة العرض أم لا:

- إذا كان كلا الرمزين يساوي 0000 فإن القطعة بالكامل داخل منطقة العرض إذا لم يكن ناتج عملية (and) بين الرمزين هو 0000 فإن القطعة خارج منطقة العرض وهو شرط لازم لكنه غير كافٍ أي يمكن أن تقع ويمكن ألا تقع. أما إذا كان ناتج عملية (and) هو 0000 فإن القطعة أو جزء من القطعة المستقيمة داخل منطقة العرض

وفي هذه الحالة نقوم بتقسيم القطعة المستقيمة إلى مستقيمين أو أكثر حسب الخطوط التي تتقاطع معها. نأخذ نقطة تقع خارج النافذة أو المنطقة (0000) هذه النقطة هي أحد طرفي القطعة المستقيمة مثلاً إذا كانت تقع أعلى النافذة فهي حتماً تتقاطع مع الخط الأعلى up حسب نقطة التقاطع ونهمل القسم الموجود بالكامل خارج النافذة، وتوجد 4-bit code للنقطة التقاطع الجديدة وهكذا حتى نحصل على الجزء المطلوب من القطعة المستقيمة أي طرفا القطعة ضمن المنطقة 0000

مثال:



سنقوم بالتطبيق على القطعة E:

أولاً نقوم بعملية AND بين رمز النقطة E أو رمز النقطة F:

$$0100 \text{ AND } 1010 = 0000$$

إذاً جزء من القطعة المستقيمة يقع داخل منطقة العرض وجزء خارج منطقة العرض

نبدأ من النقطة E والتي لها الرمز 0100 وحسب هذا الرمز فإن E تقع أسفل منطقة العرض إذاً القطعة حتماً تتقاطع مع مستقيم الـ down، ونقطة التقاطع هي F إذاً نطبق الخوارزمية على F وبما أن رمز F هو 0000، نأخذ النقطة I والتي رمزها 1010 أي تقع فوق النافذة (لنأخذ أول بت يساوي الواحد) إذاً فالقطعة المستقيمة حتماً تتقاطع مع مستقيم الـ up ونقطة التقاطع هي النقطة H تقع إلى يمين منطقة العرض والقطعة تتقاطع مع مستقيم الـ right ونقطة القاطع هي G ورمز النقطة هو 0000

إذاً حصلنا على القطعة FG وهي الجزء الواقع بالكامل داخل منطقة العرض.

ملاحظة: نسمي الحالة التي تكون في القطعة المستقيمة بكاملها داخل منطقة العرض . Trivial acceptance أي مقبولة بدون شرط، ونسمي الحالة التي تكون فيه القطعة المستقيمة غير متقاطعة مع منطقة العرض Trivial rejected

✍ إن هذا يطبق على 2D أما في حالة 3D Clipping:

كل منطقة تحوي 6 bit بدلاً من 4 bit ولدينا 27 منطقة في ثلاثة مستويات أي أصبح لدينا مناطق في الأمام (front) ومناطق في الخلف (behind):



✍ ونميز حالتين حسب نوع الإسقاط:

1. في حالة الاسقاط المتوازي:

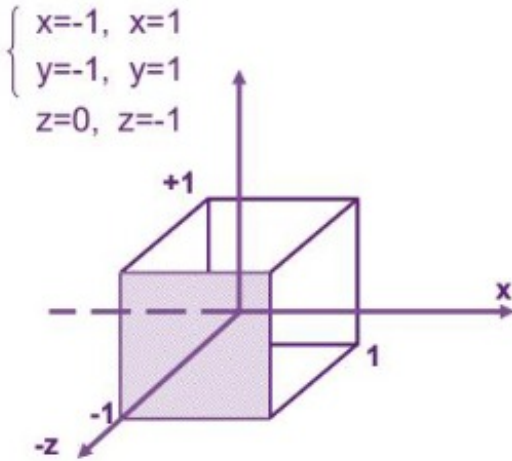
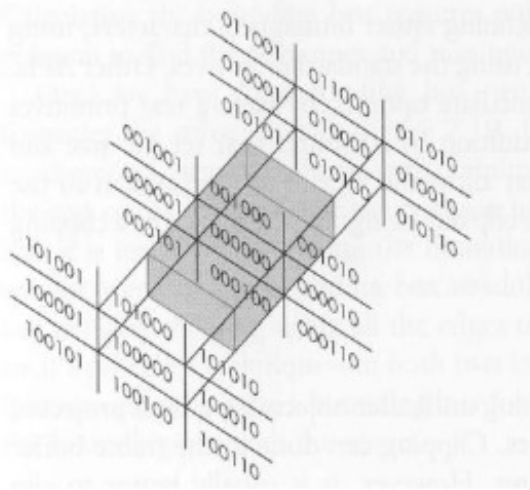
شكل منطقة العرض هو متوازي مستطيلات وبما أننا حولنا إلى الشكل القانوني أصبح بإمكاننا حساب موقع النقطة بسهولة، وتكون دلالة كل بت كالآتي:

- Bit 1 <----- النقطة فوق منطقة العرض $y > 1$
- Bit 2 <----- النقطة تحت منطقة العرض $y < -1$
- Bit 3 <----- النقطة يمين منطقة العرض $x > 1$
- Bit 4 <----- النقطة يسار منطقة العرض $x < -1$
- Bit 5 <----- النقطة خلف منطقة العرض $z < -1$
- Bit 6 <----- النقطة أمام منطقة العرض $z > 0$

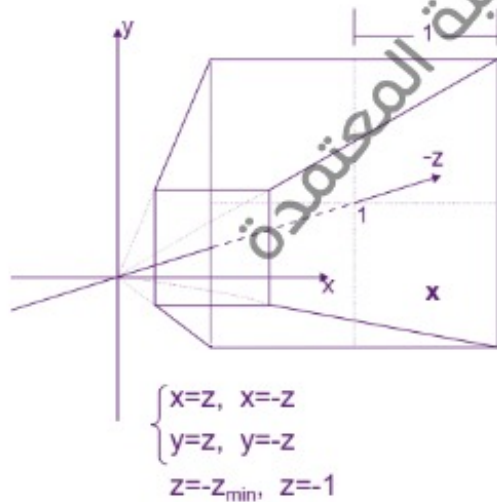
2. في حالة الاسقاط المنظوري:

شكل منطقة العرض جذع هرم لذلك المستويات الثلاث لن تكون متساوية أيضاً بعد التحويل إلى الشكل القانوني يصبح حساب موقع النقاط أسهل، وتكون دلالة كل بت كالآتي:

- Bit 1 <----- النقطة فوق منطقة العرض $y > -z$
- Bit 2 <----- النقطة تحت منطقة العرض $y < z$
- Bit 3 <----- النقطة يمين منطقة العرض $x > -z$
- Bit 4 <----- النقطة يسار منطقة العرض $x < z$
- Bit 5 <----- النقطة خلف منطقة العرض $z < -1$
- Bit 6 <----- النقطة أمام منطقة العرض $z > z_{min}$



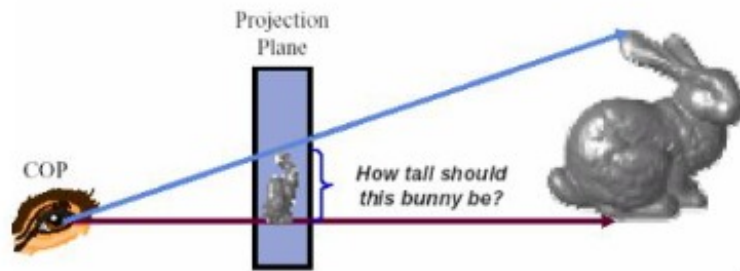
Parallel



Perspective

Perspective Projection

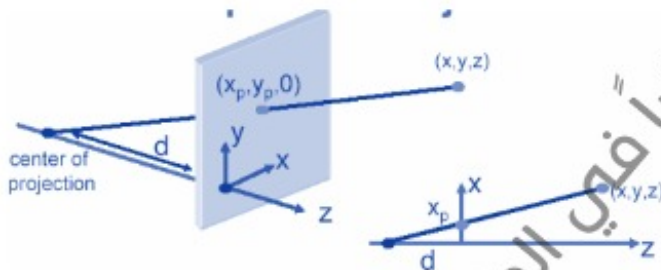
بعد القيام بالعمليات الـ clipping سنحول إلى 2D وذلك يتم عن طريق عمليات الإسقاط :



ستفرض أن:

- مستوي الإسقاط هو 2D
- الشعاع الناظم NPN هو المحور $z = (0, 0, -1)$
- V_{up} هو المحور y (أي جهة y للأعلى)
- PRP هي $(0, 0, -1)$ (هي مكان عين المشاهد أو عدسة الكاميرا وهنا نفسها (cop))

نريد حساب إحداثيات مسقط النقطة (عند أذن الأرنبي) على مستوى الإسقاط:
باستخدام تشابه المثلثات:



$$\frac{x_p}{d} = \frac{x}{z+d} \quad ; \quad \frac{y_p}{d} = \frac{y}{z+d}$$

$$x_p = \frac{d \cdot x}{z+d} \quad ; \quad y_p = \frac{d \cdot y}{z+d} \quad ; \quad z_p = 0$$

$$\therefore x_p = \frac{dx}{z/d+1} = \frac{x}{z/d+1} \quad y_p = \frac{dy}{z/d+1} = \frac{y}{z/d+1}$$

حيث d هي المسافة بين Cop ومستوي الإسقاط ويحددها المطور

مثال:

ما هو مسقط النقطة $(1, 2, 3)$ حيث $d=1$

نعوض في العلاقات السابقة:

$$x_p = \frac{1 \times 1}{3+1} = \frac{1}{4} \quad y_p = \frac{1 \times 2}{3+1} = \frac{1}{2}$$

ونعلم ان $z_p = 0$ إذا إحداثيات المسقط $(\frac{1}{4}, \frac{1}{2}, 0)$

الصبر ليس انتظاراً، بل هو القدرة على الحفاظ على الإيجابية أثناء العمل الجاد

باستخدام المصفوفات والإحداثيات المتجانسة:

$$M_{per} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & \frac{1}{d} & 1 \end{bmatrix} \quad \text{حيث } z = 0, h = \frac{1}{d}$$

h يدل على عملية الـ Scale التي حدثت بعد الإسقاط.

سنضرب المصفوفة بالنقطة للحصول على مسقط هذه النقطة على مستوى الإسقاط:

$$M_{per} P = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & \frac{1}{d} & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x \\ y \\ 0 \\ \frac{z+d}{d} \end{bmatrix}$$

$$x_p = \frac{dx}{z+d}; y_p = \frac{dy}{z+d}; z_p = 0$$

بعد الضرب علينا القسمة على $h = \frac{z+d}{d}$ للعودة للإحداثيات العادية.

ماذا عن الإسقاط المتوازي؟

في الإسقاط المتوازي العمودي لا تتغير إحداثيات x و y لذا سنركز على جعل $z=0$ فتكون المصفوفة:

$$M_{ort} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

$$p' = (x, y, 0, 1)$$

لاحظ هنا أن $h = 1$ حيث أن الإسقاط المتوازي لا يقوم

بأي عملية Scale

ملاحظة: إن مصفوفة الإسقاط المتوازي هي نفسها مصفوفة الإسقاط المنظوري ولكن نعلم أنه في الإسقاط المتوازي d تسعى للنهاية:

$$\text{When } d = \infty$$

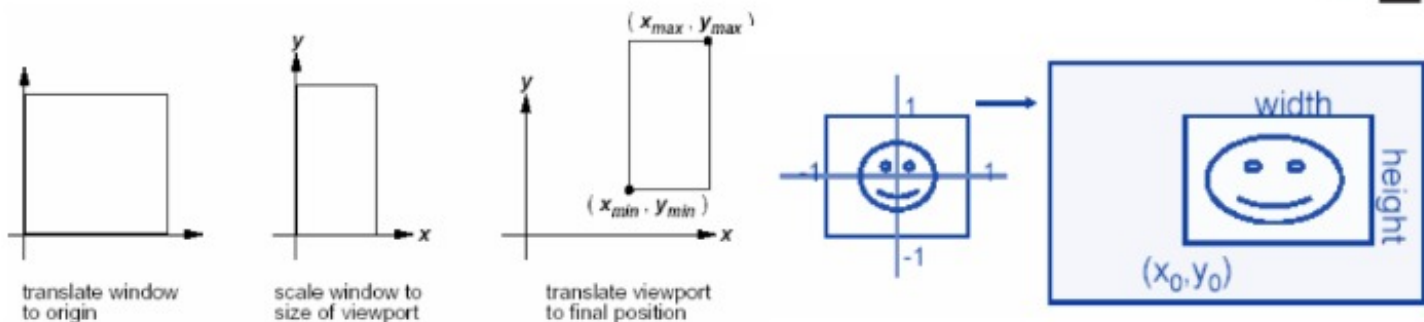
$$M_{per} \rightarrow M_{ort}$$

(الانتقال من window الى viewport: Mapping into a view port)

بعد القيام بعملية الإسقاط حولنا من 3D إلى 2D، لكن نعلم أن الإسقاط. لا يغير جملة الإحداثيات أي نحن لازلنا في جملة إحداثيات الكاميرا

إذاً علينا القيام بعملية mapping إلى الـ view port

يتم ذلك عن طريق translate الـ window لتصبح في مبدأ الإحداثيات ثم نقوم بـ scale لتصبح النافذة حجم الـ viewport ثم نقوم بـ translate إلى المكان المطلوب الرسم فيه



وتكون مصفوفة التحويل:

$$\begin{pmatrix} x_{win} \\ y_{win} \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & x_0 \\ 0 & 1 & y_0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \frac{width}{2} & 0 & 0 \\ 0 & \frac{height}{2} & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x_c \\ y_c \\ 1 \end{pmatrix}$$

Basic Raster Graphics Algorithms (2D)

ذكرنا سابقاً أن العمليات الأساسية في الغرافيكس هي:

- Modeling : وهي عملية وصف الغرض للحاسوب
- Rendering : تحويل الـ Model من 3D إلى 2D لجعله صورة تظهر على الشاشة.
- Displaying : عرض الصورة على الشاشة

إذاً بعد أن تحدثنا في المحاضرات السابقة عن عمليات الـ Modeling والـ Rendering ووصلنا إلى الـ 2D image ، كيف نقوم بعملية الرسم الحقيقي على الشاشة ؟
إن عملية الـ Displaying تتعلق بآلية عمل الشاشة.

لدينا نوعان من الشاشات:

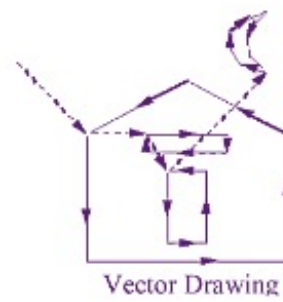
1. Vector Display:

تعتمد على مدفع الإلكترونات في آلية عملها ولا تتعامل مع الـ pixels ، أي ترسم الخطوط والمنحنيات عن طريق تحريك مدفع الإلكترونات (electron beam) حيث يزيد رسم الخط من نقطة البداية إلى نقطة النهاية ومعلومات الرسم هذه تكون مخزنة داخل file أو list

مساوئ الـ vector display:

- في حالة عدد الـ vectors الكبير أو الرسم المتحرك (animation) يصبح معدل الـ refresh عالي وعمليات مستمرة من التحديث وإعادة الرسم.
- (مثلاً إذا أردنا رسم مثلث فارغ أي غير مملوء نرسم فقط ثلاث خطوط، لكن في حال أردنا مثلث مملوء سيتطلب الكثير من الـ vectors إذاً ستبدأ الـ vectors التي رسمت بالاختفاء فنحتاج إلى عمليات redrawn .refresh
- في حالة الرسوم المعقدة يحدث Flicker في الشاشة (تذبذب أو رمش في الشاشة)

- محدودية ذاكرة تخزين أوامر و معادلات الرسم (Display list)
- صعوبة البرمجة



تقنية vector Display لا تزال تستخدم في طابعات الـ potters وفي بعض الأجهزة الطبية، لكنها لم تعد مستخدمة في شاشات العرض الحديثة. يستخدم مبدأ الـ vector في تخزين الصور، حيث يتم تخزين طريقة رسم الصورة في ملف (معادلاتها) بدلاً من تخزين الصورة كـ pixels، وتكون مساحة تخزين كملف أقل من مساحة التخزين كـ pixels، ولكن هذا فقط في حالة الصورة بعناصر واضحة، فإذا كانت الصورة معقدة يزداد عدد الأوامر في الملف ويصبح حجمه أكبر بكثير من حجم التخزين كـ pixel، ويستخدم الـ vector في تصميم الـ logos أو الـ posters حيث تكون العناصر بسيطة وواضحة وبحواف ناعمة (برنامج illustrator)

2. Raster Display:

هنا تكون الشاشة مقسمة إلى pixels، لا يهمها في آلية عملها على ما هو الشكل المراد رسمه بل تعتمد على معرفة لون ومكان الـ pixels.



outline primitives



filled primitives

يوجد في كرت الشاشة ذاكرة تسمى Refresh buffers Frame buffer كل Pixel له مكان في هذه الذاكرة، نعين الـ pixels التي نريد رسمها في الذاكرة، لتقوم الشاشة بقراءة ورسمها.

إذا عملية تحويل أي شكل إلى pixels تسمى Rasterization

تتم عملية الـ Rasterization عن طريق خوارزميات scan conversion وهي خوارزميات لمعادلات الـ primitives لتحويلها إلى pixels، وتكون مخزنة على كرت الشاشة لأنها يجب أن تكون سريعة جداً.

- On a vector device

يتم رسم خط تحريك شعاع الإلكترونات من إحداثيات البداية إلى إحداثيات النهاية. يكون هذا الخط ناعم smooth

- On raster devices

تم الرسم عن طريق خوارزميات scan conversion، ويكون الخط عبارة عن سلسلة من النقاط pixels لذا يظهر بشكل متكسر أي له مظهر الدرج staircase وتسمى ظاهرة التكسر Aliasing وسنرى كيفية معالجتها لاحقاً.

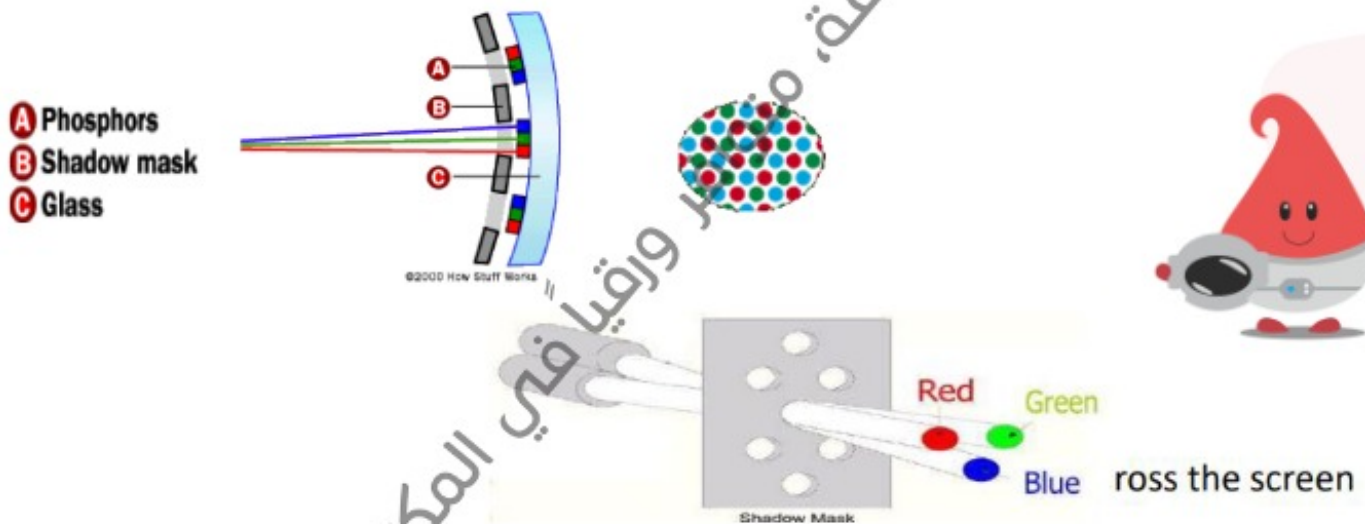
Output Graphics Devices

Cathode Ray tubes (CRTS):

تعمل شاشات CRTS عن طريق مدفع الكاترونات يطلق الكاترونات على اللوحة بلورية مدهونة بمادة فوسفورية تلمع عند اصطدام الكاترون بها، فإذا تم إطلاقه الكاترون فان هذا ال pixel مضيء لونه أبيض، وإذا لم يتم إطلاقه الكاترون فان ال pixel مظلم أي لونه أسود، وهنا كانت تستطيع اللونين الأسود والأبيض فقط .
من خلال زيادة عدد الكاترونات المطلقة في المنطقة المضاء استطعنا الحصول على تدرجات بين الأبيض والرمادي أي كلما كانت الكاترونات المطلقة أكثر اقترب اللون من الأبيض أكثر .

Color Displays:

استخدم مبدأ الدمج لظهور الألوان، حيث تم وضع ثلاث مواد مشعة في كل Pixel ، حمراء وخضراء وزرقاء، يتم إطلاقه ثلاث أشعة على كل لون ونزيد شدة الأشعة عند اللون الذي نريد أن يغلب على ال pixel ، وتقوم عين الإنسان بدمج الألوان الثلاث مع بعض ، وتم وضع حاجزين كل pixel لتجنب دمج الألوان بين ال pixels المجاورة.



إذا كيف يتم عرض الألوان في شاشة CRT؟ باستخدام مبدأ الدمج

مثلاً اللون الأصفر الحقيقي الذي نراه في الواقع له طول موجي معين، عندما تصل الموجة إلى شبكية العين يتم تفسيره على أنه لون أصفر، أما في تقنية الدمج ينتج اللون الأصفر من خلال دمج موجتي اللون الأخضر والأحمر ويتم تفسير هذا الاندماج على أنه لون أصفر

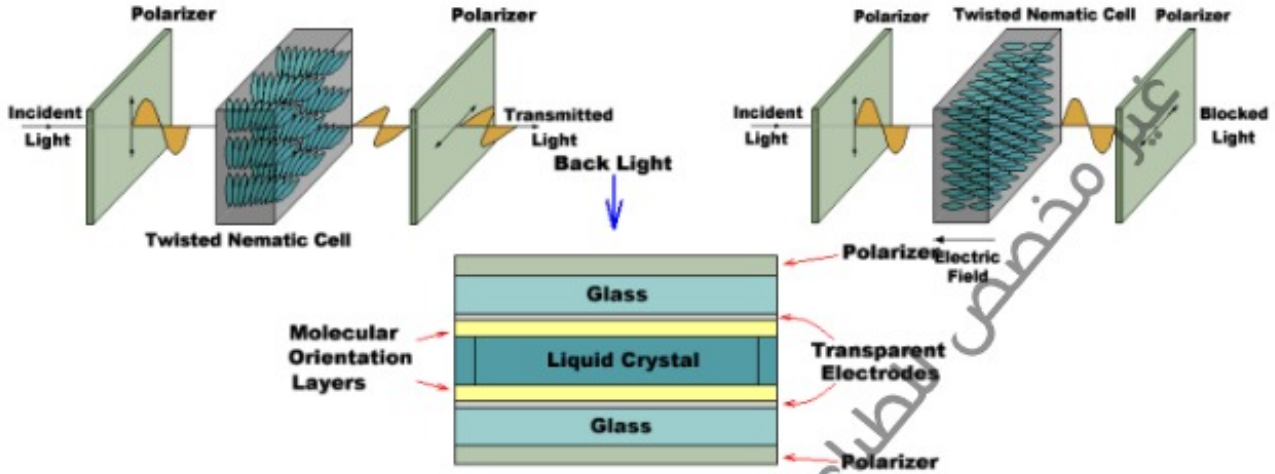
مساوئ CRTS :

- تستهلك كهرباء
- تؤذي العينين لأنها تطلق الكاترونات و أشعة فوق بنفسجية UV
- وزنها ثقيل

شاشات CRTS تعمل بطريقة Vector Displays

Liquid Crystal Displays (LCDs):

في هذه الشاشة لدينا منبع ضوئي نضع أمامه مادة سائلة كريستالية (liquid crystal diod) هذا الديود إما أن يمرر الضوء أولاً يمرره،
مثلاً إذا أردنا Pixel بلون أسود لا نمرر الضوء، أما إذا أردنا pixel بلون أبيض نمرر الضوء،

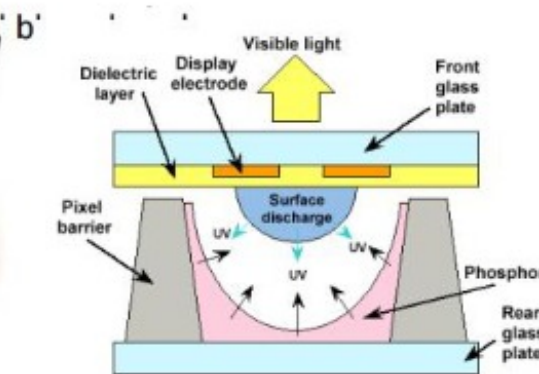


مسائري LCDs

- ألوانها باهتة لأنها لا تشع الضوء بل تمرره.
- لا يمكن رؤية الصورة واضحة سوى عند الوقوف أمامها مباشرة ، لأن الضوء الذي لا يمرره الديود يقوم بإزاحته أو كسره لذا تبدو الصورة مشورة عند النظر إلى الشاشة من زاوية جانبية .
- لعرض ألوان تم وضع ثلاث فلاتر أمام ثلاث ديودات بألوان RGB (أحمر - أخضر - أزرق) يستخدم فيه أيضاً مبدأ الدمج لعرض ألوان مختلفة.

Plasma Display Panels (PDPS):

كل pixel يعمل كنيون ، حيث يحوي كل Pixel أنبوب غازي ليشتع بثلاث ألوان RGB إذا كل Pixel عبارة عن منبع ضوئي يمكن إضاءته أو إطفائه.
حجم الـ pixel هنا كبير لأنه يحوي نيون، لذلك تكون شاشات PDPS شاشات عرض كبيرة كما في شاشات السينما.



Light-Emitting Diodes (LED):

بكل pixel يوجد ثلاث LEDs بألوان RGB ويمكن إضاءة كل LED وإطفائه ولكن اذا تعطل الـ LED في الـ pixel لا يمكن إصلاحه ويسمى Dead pixel
هذه الشاشة هي المستخدمة حالياً في الحواسيب والهواتف المحمولة حيث يمكن تصغير الـ LED.

Scan Commenting primitives

سنحدث عن خوارزميات رسم مستقيم ودائرة في raster display:

Scan Converting Lines:

لدينا شبكة من الـ pixels ولدينا pixel البداية و Pixel النهاية، ونريد إيجاد باقي الـ Pixels لرسم المستقيم.
ذكرنا سابقاً أن خوارزميات الـ scan conversion يجب أن تكون سريعة جداً لذا لدينا بعض الشروط وهي:

- استخدام عمليات الجمع بدلاً من الضرب والقسمة
- التعامل مع أعداد integer بدلاً من float.
- عدم استخدام التوابع المساعدة لأن استغائها مكلف (توابع التقريب - القيمة المطلقة)
- نعلم أن معادلة المستقيم هي:

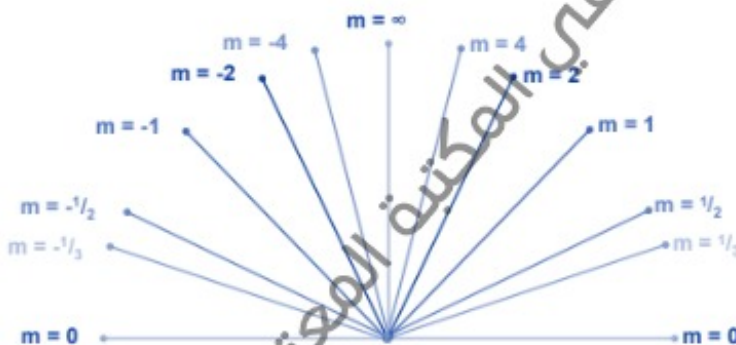
$$Y = mx + b$$

$$m = \frac{y_2 - y_1}{x_2 - x_1}$$
حيث هو ميل المستقيم:

$$b = y - mx$$
مقدار الإزاحة على المحور y (التقاطع مع y):
- إحداثيات كل Pixel هي integer (أي كل من x, y هي أعداد صحيحة) ولا نتعامل مع جزء
- يجب أن يكون ميل المستقيم بين $(0 < m < 1)$

حيث أن ميل أي مستقيم هو:

إذا كان الميل أكبر من 1 ($m > 1$) نبدل بين x و y
إذا كان الميل سالب نرسم المستقيم بميل موجب ونقلبه.



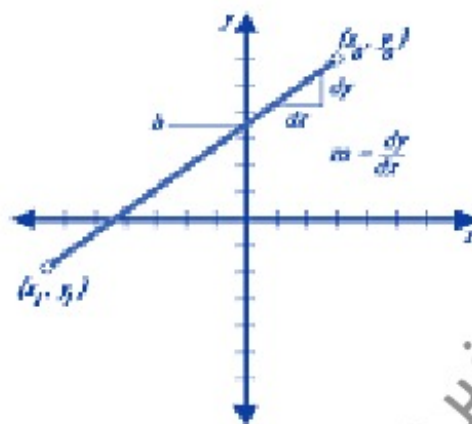
Naive algorithm

بتعويض قيمة كل x بشكل متتال بالترتيب أي نزيد 1 لـ x في كل مرة في معادلة المستقيم نحصل على قيمة y ونقوم بتقريبها إذا كانت غير صحيحة فنصل على إحداثية كل pixel في المستقيم


```

Void Line (
    int x0, int y0,
    int x1, int y1)
{
    int dx = x1 - x0;
    int dy = y1 - y0;
    WritePixel(x0, y0);
    if (dx != 0)
    {
        float m = (float) dy / (float) dx;
        float b = y0 - m * x0;
        dx = (x1 > x0) ? 1 : -1;
        while (x0 != x1)
        {
            x0 += dx;
            y0 = Round(m * x0 + b);
            WritePixel(x0, y0);
        }
    }
}

```



ولكن هذه الخوارزمية لها الكثير من المشاكل :

■ تحتوي قيم float

■ تحوي ضرب وقسمة

■ تستدعي تابع التقريب Round

إذا تعطي نتيجة صحيحة لكن غير فعالة وبطيئة جداً.

Basic Incremental Algorithm

أو تعرف بخوارزمية Digital Differential Analyzer (DDA)

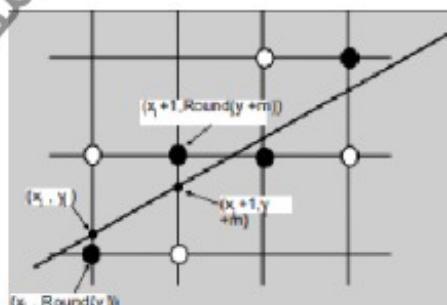
سنستفيد هنا من فكرة أن مقدار التغير في x في المستقيم هو واحد $Dx=1$ للتخلص من الضرب فنلاحظ أن :

$$Y_{i+1} = m_{xi+1} + b = m (xi + Dx) + b = yi + m Dx$$

$$\text{if } Dx = 1 \text{ Then } y_{i+1} = y_i + m$$

$$x_{i+1} = x_i + 1 \quad y_{i+1} = y_i + m$$

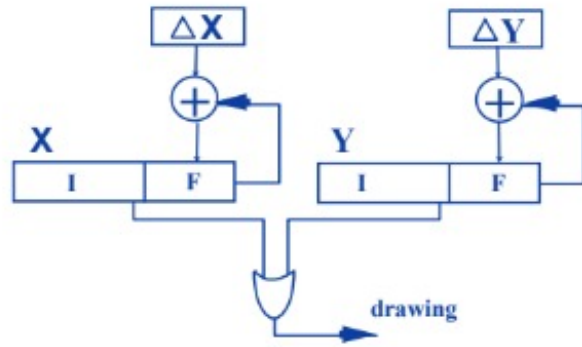
أي أن كل زيادة x بمقدار 1 يقابلها زيادة على y بمقدار ثابت m.



Avoid Rounding

هذه الخوارزمية تساعدنا في التخلص من التقريب بسهولة وذلك لتوفير الوقت والتخلص من عمليات تكرار رسم نفس النقطة أكثر من مرة.

تجعل لكل من x و y قسمين، قسم صحيح (Integer) I وقسم كسري (Fractional) F



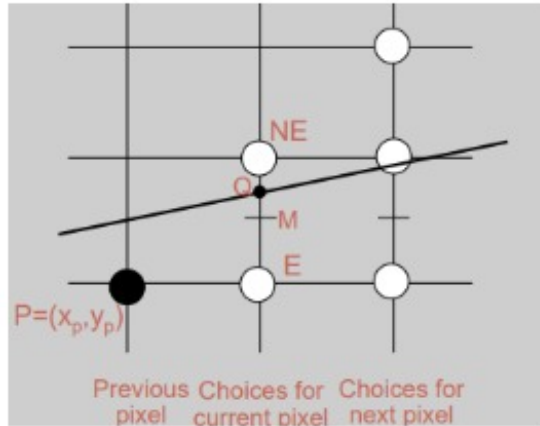
نضيف قيمة التزايد على الجزء الكسري F
عندما تتجاوز قيمة الجزء الكسري F الواحد (overflowed)،
تتم زيادة الجزء الصحيح | الموافق ورسم النقطة.
هذه الخوارزمية جيدة لكن عمليات التقريب
تعطي نتيجة غير دقيقة أي نسبة الخطأ في كبيرة

MidPoint Line Algorithm

أو تعرف بخوارزمية (BRESEMPHAM):

مبدأ الخوارزمية هي تحويل المسألة من مسألة حساب إلى مسألة اختيار. فنحن فرضنا مسبقاً أن ميل المستقيم بين 0 و 1، لذا عند زيادة x بمقدار واحد نحن أمام خيارين لـ y إما أن تزيد y واحد أو أن تبقى ثابتة ولا يوجد خيار آخر.

لدينا في الشكل مجموعة من النقاط:



- النقطة p هي الـ Pixel الذي اخترناه مسبقاً ليكون من المستقيم.
- النقطة E النقطة التي تقع على يمين النقطة p
- النقطة NE هي النقطة التي تقع أعلى يمين النقطة p
- النقطة M تقع منتصف E, NE

إذا نريد اختيار النقطة التالية للنقطة p وهي إما يمين النقطة p أو أعلى يمين النقطة p، فإذا كان المستقيم يمر من فوق النقطة

M نختار النقطة NE

وإذا كان أسفل النقطة M نختار النقطة E

إذاً بتعويض النقطة M في معادلة المستقيم $F(x, y)$ نحصل على:

إما $F(x, y) > 0$ واقعة تحت المستقيم.

أو $F(x, y) < 0$ واقعة فوق المستقيم أي معادلة من الشكل:

$$ax + by + c = 0$$

معادلة المستقيم $y = \frac{dy}{dx}x + b$ يمكن ان نكتبها بالشكل:

$$F(x, y) = dy \cdot x - dx \cdot y + b \cdot dx = 0$$

علينا التخلص من عمليات الضرب:

يمكننا الحصول على M التالية لكل M بزيادة 1 لـ x و $\frac{1}{2}$ لـ y أي إذا كانت $M1(x_m, y_m)$ فإن $M2(x_{m+1}, y_{m+\frac{1}{2}})$

$$d = F(M) = F(x_p + 1, y_p + \frac{1}{2}) = a(x_p + 1) + b(y_p + \frac{1}{2}) + c$$

$d > 0$ نختار NE

$d < 0$ نختار E

✚ إذا اخترنا النقطة E:

اختيار النقطة E يعني Xm سيزداد بمقدار 1:

$$d_{new} = F(xp + 2, yp + \frac{1}{2}) = a(xp + 2) + b(yp + \frac{1}{2}) + c$$

$$d_{old} = F(xp + 1, yp + \frac{1}{2}) = a(xp + 1) + b(yp + \frac{1}{2}) + c \quad \text{ولدينا}$$

بالمقارنة بين d_{old} و d_{new} نستنتج أن:

$$\Delta E = d_{new} - d_{old} = a$$

ونعلم أن: $\Delta E = dy$ إذاً $a = dy$

أي أنه في كل مرة نختار النقطة E نقوم بزيادة له بالمقدار:

$$incrE = a = dy$$

✚ إذا اخترنا النقطة NE:

اختيار النقطة NE يعني زيادة كل من ym بمقدار 1:

$$d_{new} = F\left(xp + 2, yp + \frac{3}{2}\right) = a(xp + 2) + b\left(yp + \frac{3}{2}\right) + c$$

$$d_{old} = F\left(xp + 1, yp + \frac{1}{2}\right) = a(xp + 1) + b\left(yp + \frac{1}{2}\right) + c$$

بالمقارنة بين d_{old} و d_{new} نستنتج أن:

$$NE = d_{new} - d_{old} = a + b$$

نعلم أن $a = dy$ و $b = -dx$

$$\Delta NE = dy - dx \quad \text{إذاً:}$$

أي أنه في كل مرة نختار النقطة NE نقوم بزيادة d بالمقدار:

$$incr NE = dy - dx$$

Initializing d

علينا حساب قيمة d الابتدائية أي $F(M)$ الابتدائية:

إذا كان أول Pixel هو (x_0, y_0) فإن أول d يتم حسابها هي: $(x_0 + 1, y_0 + \frac{1}{2})$

إذاً:

$$F(x_0 + 1, y_0 + \frac{1}{2}) = a(xp + 1) + b(yp + \frac{1}{2}) + c$$

$$= ax_0 + by_0 + c + a + \frac{b}{2} = F(x_0, y_0) + a + \frac{b}{2}$$

لدينا $F(x_0, y_0) = 0$ لأن نقطة البداية تحقق معادلة المستقيم.

إذاً:

$$dinial = a + \frac{b}{2} = dy - \frac{dx}{2}$$





وللتخلص من عملية القسمة على 2 نضرب معادلة المستقيم بـ 2 :

$$F(x, y) = 2(ax + b + c)$$

ويصبح:

$$d_{init} = 2dy - dx$$

يصبح أيضاً:

$$\Delta E = 2dy$$

$$\Delta E = 2(dy - dx).$$

```

Void MidPointLine (
    int x0, int y0,
    int x1, int y1,
    int value)
{
    int x;

    int dy = y1 - y0;
    int dx = x1 - x0;
    int d = 2*dy-dx;
    int incrE = 2*dy;
    int incrNE = 2*(dy-dx);
    int x=x0;
    int y = y0;
    WritePixel (x, y, value);
    While (x < x1) {
        if (d <= 0) {
            d+=incrE;
            x++;
        } else {
            d+=incrNE;
            x++;
            y++;
        }
        WritePixel (x,y, value)
    } /* While */
} /* MidPointLine */

```

/* Assumes -1 <= m <= 1, x0 < x1 */
 /* Left end point */
 /* Right end point */
 /* Value to place in line's pixels */
 /* x runs from x0 to x1 in unit increments */
 /* Initial value of d */
 /* Increment used for move to E */
 /* Increment used for move to NE */
 /* The start pixel */
 /* Choose E */
 /* Choose NE */
 /* The selected pixel closest to the line */

نسبة الخطأ في هذه الخوارزمية $\geq \frac{1}{2}$
 كل الحسابات فيها تتم باستخدام الأعداد الصحيحة، واستبدلنا عمليات الضرب والقسمة لعمليات الجمع والطرح.

مثال :

نريد رسم خط باستخدام خوارزمية MidPoint (أو خوارزمية BRESEMHAM)

نقطة بداية (Start point) هي: $S = (2, 4)$

نقطة نهايته (End point) هي: $E = (10, 7)$

الحل

لدينا $\Delta x = 8$ و $\Delta y = 3$

نحسب m فنجد: $0 < m = \frac{\Delta y}{\Delta x} = \frac{3}{8} \approx 0.4 \leq 1$

نحسب قيمة d الابتدائية: $d_{init} = 2dy - dx = -2$

نحسب مقدار الزيادة (ΔE) عند الانتقال الى النقطة أي عندما $d < 0$:

$$\Delta E = 2 dy = 6$$

نحسب مقدار الزيادة (ΔE) عند الانتقال إلى النقطة NE أي عندما $d \geq 0$:

$$\Delta NE = 2(dy - dx) = -10$$

نقطة البداية هي (2,4) وقيمة dinit أصغر من الصفر إذاً نختار البكسل على يمين النقطة الحالية أي البكسل في النقطة (3,4)

نقوم بإضافة (ΔE) إلى قيمة dinit فتصبح $d = 4$ أكبر من الصفر إذاً نختار البكسل في أعلى اليمين أي (4,5) وهكذا حتى نصل إلى نقطة النهاية.

x	y	D	أي نقوم بزيادة قيمة y بمقدار 1 تبعاً لقيمة dold إذا كانت أكبر من الصفر
2	4	-2	
3	4	4	
4	5	-6	
5	5	0	
6	6	-10	
7	6	-4	
8	6	2	
9	7	-8	
10	7		

Scan Converting Circle

لدينا أيضاً خوارزميات لرسم دائرة:

Naive Algorithm

نعلم أن معادلة دائرة مركزها (0,0) هو ونصف قطرها R هي:

$$x^2 + y^2 = R^2$$

$$y = f(x) \Rightarrow y = \pm \sqrt{R^2 - x^2}$$

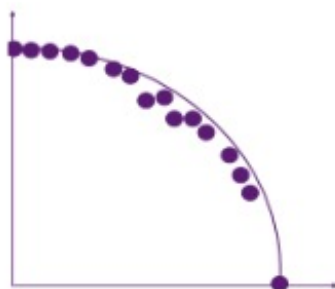
نزيد قيمة x من 0 إلى R ونأخذ القيم الموجبة لـ y وذلك لرسم الربع الأول من الدائرة، ونرسم الأرباع البقية بالتناظر. ولكن هذه الطريقة غير فعالة:

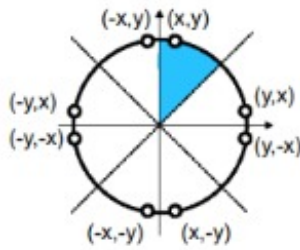
لوجود عمليات الضرب والجذر

تسبب فراغات كبيرة بين النقاط تزداد مع اقتراب قيم x من R.

يمكن تجنب هذه الفراغات باستخدام الاحداثيات القطبية ولكن

إحداثيات الشاشة هي إحداثيات ديكرتية

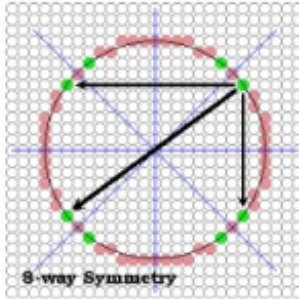




Eight-Way Symmetry

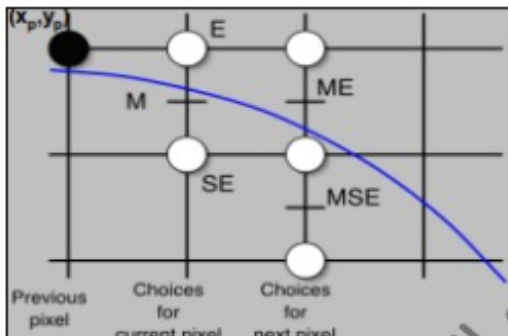
يمكن الاستفادة من التناظر لتحسين الخوارزمية وذلك برسم ثمن واحد من الدائرة.

فإذا كانت النقطة (x, y) تنتمي إلى الدائرة يمكننا ببساطة حساب سبع نقاط أخرى تنتمي للدائرة



MidPoint Circle Algorithm:

نفس مبدأ خوارزمية Midpoint للخطوط المستقيمة لكن هنا الاختيار بين النقطة التي تقع إلى اليمين E أو النقطة التي تقع إلى اليمين والأسفل (SE) معادلة الدائرة:



$$F(x, y) = x^2 + y^2 - R^2$$

$F(x, y) = 0$ النقطة تقع على الدائرة.

$F(x, y) > 0$ النقطة تقع خارج الدائرة.

$F(x, y) < 0$ النقطة تقع داخل الدائرة.

إذا كانت M خارج الدائرة نختار النقطة SE وإذا كانت M داخل الدائرة.

نختار النقطة E

وكما في حالة المستقيم لدينا:

$$F(M) = F(xp + 1, yp - \frac{1}{2})$$

$$d = F(xp + 1, yp - \frac{1}{2}) = (xp + 1)^2 + (yp - \frac{1}{2})^2 - R^2$$

$d < 0$ تختار النقطة E

$d > 0$ تختار النقطة SE

إذا اخترنا النقطة E:

$$d_{new} = F(xp + 2, yp - \frac{1}{2}) = (xp + 2)^2 + (yp - \frac{1}{2})^2 - R^2$$

$$d_{old} = F(xp + 1, yp - \frac{1}{2}) = (xp + 1)^2 + (yp - \frac{1}{2})^2 - R^2$$

$$d_{new} = d_{old} + (2xp + 3)$$

$$\Delta E = d_{new} - d_{old} = 2xp + 3.$$

$$\Delta E = 2xp + 3$$

✎ إذا اخترنا النقطة NE:

$$d_{new} = F(xp + 2, y - \frac{3}{2}) = (xp - 2)^2 + (yp - \frac{3}{2})^2 - R^2$$

$$\Delta E = 2xp - 2yp + 5 \Rightarrow \Delta E = 2(xp - yp) + 5.$$

Initializing d:

أول pixel في الدائرة في النقطة (0, R)

إذاً الـ Pixel الثاني في النقطة (1, R - 1/2)

$$\Rightarrow F(1, R - \frac{1}{2}) = d_{init} = \frac{5}{4} - R$$

ويمكن استبدالها بـ: $d_{init} = 1 - R$ لتجنب وجود أعداد كسرية

```

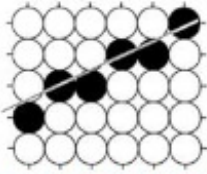
Void MidPointCircle ( /* Assumes center of circle is at origin */
    int radius,
    int value) /* Value to place in line's pixels */
{
    int x=0;
    int y = radius;
    double d=5.0/4.0 -radius; /* Initial value of d */
    CirclePoints(x,y,value);
    While (y > x) {
        if (d<=0) /* Choose E */
            d+=2.0*x+3.0;
        else {
            d+=2.0*(x-y)+5.0; /* Choose NE */
            y--;
        }
        x++;
        CirclePoints(x,y,value);
    } /* While */
} /* MidPointCircle */

```

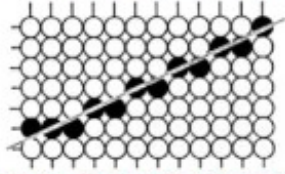
Aliasing and anti-Aliasing

ذكرنا أن الخطوط المرسومة بالـ pixels تظهر بشكل منكسر وقلنا أن هذه الظاهرة تسمى بظاهرة الدرج المنكسر Aliasing أو Staircase كما أن عمليات التقريب إلى أقرب Pixel التي رأيناها في الخوارزميات السابقة تزيد من هذه الظاهرة

- نعلم أن الصورة في الحاسوب موجودة بشكل متقطع pixels وليس بشكل مستوي
- درسنا سابقاً في مادة الاتصالات الرقمية أنه إذا قمنا بتقطيع إشارة الصوت يكون تردد التقطيع أكبر من ضعفي أعلى تردد موجود بالإشارة وذلك للحفاظ على الإشارة من الضياع
- كل ما طبقناه سابقاً على الصوت يمكن تطبيقه على الصورة.



standard midpoint line display



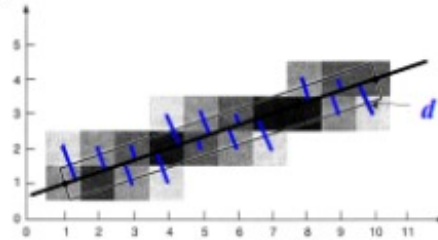
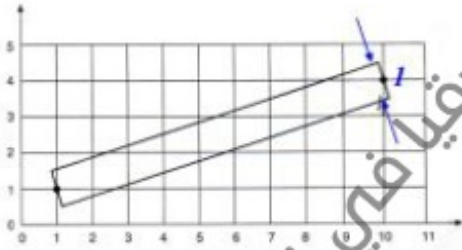
Same line display with twice resolution

إذاً يمكن زيادة دقة الصورة بزيادة تردد التقطيع أي بزيادة عدد البكسلات
تردد عالي: بكسل أبيض بجانبه بكسل أسود
ولكن زيادة الدقة بزيادة عدد البكسلات ليس حل عملي، لأنه يؤدي
إلى زيادة الذاكرة المطلوبة وزيادة كلفة خوارزميات
scan conversion ، وهذا يزيد الضغط على كرت
الشاشة وزيادة استهلاك الطاقة.

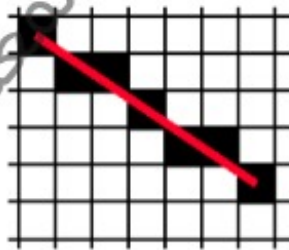
إذاً سنفكر في حل آخر وهو خفض تردد الصورة وذلك بإضافة بكسلات بتدرجات رمادية.
مثلاً أبيض بجانبه بكسل رمادي: تردد أقل.
وللقيام بذلك لدينا طريقتان:

Un - Weighted Area Sampling

نفرض أنه للمستقيم سماكة 1 بكسل (أي عرضه لا يتناهي إلى الصفر). إذاً يوجد مساحة تقاطع بين المستقيم وكل Pixel
ويتم إعطاء كثافة لونية (intensity) لكل pixel حسب التي يشغلها هذا ال Pixel من المستقيم، فالبكسل الذي يشغل
أكبر مساحة يكون لونه أسود، والبكسل الذي يشغل مساحة أقل يكون لونه رمادي، أما البكسل خارج المستقيم يكون لونه
أبيض .



لاحظ أن كثافة اللون intensity تتناسب طردياً مع المساحة التي يشغلها ال pixel. من المستقيم، مثلاً ال pixel في
النقطة (2,1) 70% من مساحته داخل المستقيم ولونه أسود وال Pixel في النقطة (2,2) فقط 25% من مساحته داخل
المستقيم ولونه رمادي.



مشكلة هذه الطريقة أنها تعتمد على المساحة التي يشغلها المستقيم من البكسل فقط دون اعتبار موقع التقاطع بينهما
فيمكن لبكسل على حافة المستقيم أن يكون له نفس لون بكسل مركزه ضمن المستقيم ويشغلان نفس المساحة .

Weighted Area Sampling

هنا لن نعتمد فقط على المساحة التي يشغلها البكسل من المستقيم، بل سنأخذ أيضاً قرب مركز البكسل من الخط، فالمساحة الصغيرة القريبة من الخط سيكون لها لون أغمق من المساحة الكبيرة الواقعة على بعد أكبر من الخط.

- سنعتبر عن ذلك رياضياً من خلال تابع تكامل للمساحة بدلالة المسافة.
 - نعرف التابع $W(x, y)$ والذي يمثل البعد بين الخط ومنتصف الـ Pixel الواقع في (x, y)
 - نعرف المساحة التي يغطيها المستقيم من الـ pixel : $\Delta A = \int dA$ حيث $dA \rightarrow \infty$
 - نعرف $Ws = \int W(x, y) dA$ ويكون $1 \geq Ws \geq 0$
 - وتكون كثافة لون الـ pixel:
- $I = Ws I_{max}$ حيث هي الكثافة العظمى للون البكسل (أبيض)

في حالة un-weighted Area يكون التابع المسافة $W(x, y)$ ثابت

ملاحظة: في OpenGL لا نستعمل اللون الرمادي للبكسل بل نحصل عليه عن طريق زيادة شفافية الـ pixel الذي لونه أسود، وذلك أدق ليندمج مع الرسم خلفه اذا تم رسم شيء وراءه، أي لكي لا يظهر الـ pixel الرمادي فقط ويزيد التكرار

النهـاية

الحياة بسيطة نحن من نعقدّها،

تعشق شيء، حافظ عليه،

تريد شيء، اتعب من أجل الحصول عليه



المكتبة المعتمدة